CNN: PyTorch 코드 읽기

`nn.Conv2d` · `nn.MaxPool2d` · `nn.Flatten` · `mnist_cnn_torch.py`

딥러닝개론및연습 · 2026년 2학기

이번 시간의 핵심 질문

이번 시간의 핵심 질문

- 합성곱 계층과 풀링 계층을 PyTorch로는 어떻게 쓸까?
- 이미지를 784개로 펼치지 않고 넣으려면 텐서의 모양을 어떻게 해야 할까?
- 완전연결 신경망의 PyTorch 코드에서 **무엇만 바꾸면** CNN이 될까?
- 같은 코드를 GPU에서 돌리면 얼마나 빨라질까? 결과는 똑같을까?

$\Rightarrow$ **`mnist_cnn_torch.py`를 처음부터 끝까지 읽는다**

이번 시간부터는 PyTorch 코드로

- 지금까지는 계층의 `forward`와 `backward`를 NumPy로 **직접** 만들었다
  - 원리를 이해하기 위해서였다. 합성곱 계층도 노트에서 `im2col`로 직접 구현했다
- 이제부터는 **PyTorch 코드**를 중심으로 읽는다
  - 계층이 많아지고 데이터가 커질수록 직접 짠 코드는 느리고, 틀리기 쉽다
  - 연구와 산업에서 쓰는 코드는 대부분 PyTorch로 되어 있다
  - 같은 코드가 GPU에서 그대로 돈다
- 그래도 우리는 각 줄 뒤에서 **무엇이 계산되는지** 알고 있다: 합성곱, 풀링, 역전파, Adam

이번 시간의 코드: `CNN/code/mnist_cnn_torch.py` (90줄), 작은 예제 모음 `cnn_torch_demo.py`

학습목표

1. 이미지 미니배치를 (N, C, H, W) 모양의 텐서로 만들 수 있다
2. `nn.Conv2d`의 인자, 가중치의 모양, 출력 크기를 설명할 수 있다
3. `nn.MaxPool2d`와 `nn.Flatten`이 하는 일을 설명할 수 있다
4. 모델의 각 계층을 지날 때 텐서의 모양과 매개변수 수를 계산할 수 있다
5. 완전연결 신경망 코드와 CNN 코드의 **같은 점과 다른 점**을 설명할 수 있다
6. 학습한 첫 번째 합성곱 계층의 필터를 꺼내 그릴 수 있다

목차

완전연결 신경망 코드에서 무엇이 바뀌나

데이터: (N, C, H, W) 텐서

새 계층 세 가지

모델: `nn.Sequential`

학습과 평가

학습한 필터 보기

정리와 실습

1

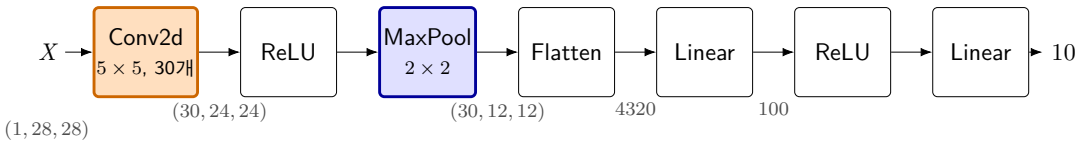
완전연결 신경망 코드에서 무엇이 바뀌나

바뀌는 것은 세 곳뿐

	완전연결 (`mnist_torch.py`)	CNN (`mnist_cnn_torch.py`)
입력의 모양	`(N, 784)`: 한 줄로 펼침	`(N, 1, 28, 28)`: **이미지 모양 그대로**
모델	`Linear`, `ReLU`, ⋯	`Conv2d`, `MaxPool2d`, `Flatten` + `Linear`
최적화 기법	SGD 또는 AdamW	Adam
데이터 분할	`mnist_data.py`: 훈련 55000 / 검증 5000 / test 10000 (같음)	
손실 함수	`nn.CrossEntropyLoss` (같음)	
학습 루프	순전파 → 손실 → `zero_grad` → `backward` → `step` (같음)	
평가	`@torch.no_grad()`, `model.eval()`, `argmax` (같음)	

- PyTorch 입문에서 말한 그대로: **모델이 바뀌어도 학습 루프는 한 글자도 바뀌지 않는다**
- 그래서 이번 시간의 중심은 **입력의 모양**과 **세 가지 새 계층**이다

신경망의 구조: SimpleConvNet



데이터 하나의 모양 (미니배치 차원 N은 생략)

- 노트에서 NumPy로 만든 `SimpleConvNet`과 **같은 구조**
- 합성곱 $\rightarrow$ 풀링으로 이미지의 **특징**을 뽑고, 펼친 뒤 완전연결 계층 두 개로 분류한다
- 마지막 `Linear`의 출력 10개가 점수(logits). softmax는 손실 함수 안에서 계산된다

2

데이터: (N, C, H, W) 텐서

준비와 데이터 불러오기

```
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

from mnist_data import load_mnist_split

FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "figures")
os.makedirs(FIG_DIR, exist_ok=True)

torch.manual_seed(0)
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"device: {device}")

x_train, t_train, x_val, t_val, x_test, t_test = load_mnist_split()   # 55000 / 5000 / 10000
```

- 지난번 `mnist_torch.py`와 **같다**. `time`은 epoch마다 걸린 시간을 재려고
- `mnist_data.py`는 같은 폴더에 복사해 둔 학습 관련 기술 주차의 모듈. 넘파이 배열 (`N`, 784)와 one-hot 정답을 준다

이미지 모양으로 되돌리기: (N, 784) → (N, 1, 28, 28)

```
def to_dataset(x, t):
    """images (N, 784) -> (N, 1, 28, 28) tensors, one-hot labels -> class indices"""
    return TensorDataset(torch.tensor(x, dtype=torch.float32).reshape(-1, 1, 28, 28),
                         torch.tensor(np.argmax(t, axis=1)))
```

```
x = torch.arange(2 * 784, dtype=torch.float32).
     reshape(2, 784)
print(tuple(x.shape), "->", tuple(x.reshape(-1, 1,
     28, 28).shape))
```

```
(2, 784) -> (2, 1, 28, 28)
```

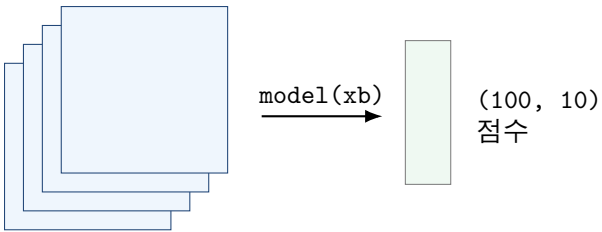
차원	뜻
N	데이터(이미지) 수
C	채널 수 (흑백 1, 컬러 3)
H	높이 (28)
W	너비 (28)

- PyTorch의 합성곱 계층은 **4차원** 텐서 (N, C, H, W)를 받는다. 노트의 NumPy 구현과 같은 순서
- `reshape(-1, 1, 28, 28)`: `-1`은 "나머지에서 계산". 784개를 다시 28×28로 접는다 (펼쳤던 순서 그대로)
- 정답은 지난번처럼 `argmax`로 클래스 번호 `(N,)`로

DataLoader

```
train_loader = DataLoader(to_dataset(x_train, t_train), batch_size=100, shuffle=True)
val_loader = DataLoader(to_dataset(x_val, t_val), batch_size=1000)
test_loader = DataLoader(to_dataset(x_test, t_test), batch_size=1000)
```

- 지난번과 같다. 다만 꺼내는 미니배치의 모양이 다르다:
  - `xb: (100, 1, 28, 28)`, `yb: (100,)`
- 훈련: $55000/100 = 550$개의 미니배치, 섞어서. 검증과 test: 1000개씩, 섞지 않고



3

새 계층 세 가지

nn.Conv2d: 합성곱 계층

```
nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)
```

```
conv = nn.Conv2d(1, 30, kernel_size=5)
print(tuple(conv.weight.shape),
      tuple(conv.bias.shape),
      sum(p.numel() for p in conv.parameters()))
```

```
(30, 1, 5, 5) (30,) 780
```

- `in_channels` C: 입력 채널 수 (1)
- `out_channels` FN: **필터 수** = 출력 채널 수 (30)
- `kernel_size`: 필터 크기 ($5 = 5 \times 5$)

- 가중치 `weight`: $(FN, C, FH, FW) = (30, 1, 5, 5)$. 노트의 NumPy 구현과 **같은 순서**
- 편향 `bias`: 필터마다 하나 $\rightarrow (30,)$
- 매개변수: $30 \times 1 \times 5 \times 5 + 30 = 780$. 입력 이미지의 크기($28 \times 28$)와 **상관없다**
  - 같은 필터를 이미지의 모든 위치에 쓰기 때문 (가중치 공유)

출력 크기: 패딩과 스트라이드

$$OH = \frac{H + 2P - FH}{S} + 1, \qquad OW = \frac{W + 2P - FW}{S} + 1$$

```
img = torch.randn(1, 1, 28, 28)
for k, s, p in [(5, 1, 0), (5, 1, 2), (3, 2, 1)]:
    out = nn.Conv2d(1, 30, kernel_size=k, stride=s, padding=p)(img)
    print(f"kernel {k}, stride {s}, padding {p}: (1, 1, 28, 28) -> {tuple(out.shape)}")
```

```
kernel 5, stride 1, padding 0: (1, 1, 28, 28) -> (1, 30, 24, 24)
kernel 5, stride 1, padding 2: (1, 1, 28, 28) -> (1, 30, 28, 28)
kernel 3, stride 2, padding 1: (1, 1, 28, 28) -> (1, 30, 14, 14)
```

- 식으로 확인: $\frac{28-5}{1} + 1 = 24$, $\frac{28+4-5}{1} + 1 = 28$ (크기 유지), $\frac{28+2-3}{2} + 1 = 14.5 \to 14$ (내림)
- `nn.Conv2d(...)(img)`: 계층을 만들고 **바로** 입력을 넣었다. 계층 객체는 함수처럼 부를 수 있다
- 출력의 채널 수는 항상 필터 수(30). 미니배치 차원(1)은 그대로

손으로 계산한 예제와 비교

```
X = torch.tensor([[1., 2., 3., 0.],
                  [0., 1., 2., 3.],
                  [3., 0., 1., 2.],
                  [2., 3., 0., 1.]]).reshape(1, 1, 4, 4)
F = torch.tensor([[2., 0., 1.],
                  [0., 1., 2.],
                  [1., 0., 2.]]).reshape(1, 1, 3, 3)
small = nn.Conv2d(1, 1, kernel_size=3, bias=False)
with torch.no_grad():
    small.weight.copy_(F)       # put our filter into the
     layer
print(small(X).detach().reshape(2, 2))
```

```
tensor([[15., 16.],
        [ 6., 15.]])
```

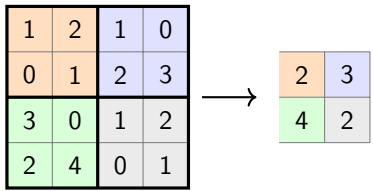
- CNN 시간의 예제: 입력 4×4, 필터 $3 \times 3 \to$ 출력 2×2
- 손으로 계산한 답 $\begin{pmatrix} 15 & 16 \\ 6 & 15 \end{pmatrix}$과 **같다**
- `bias=False`: 편향 없이
- `weight.copy_(F)`: 가중치에 우리 필터를 **제자리에서** 복사
- `torch.no_grad()`: 가중치를 직접 바꾸는 것은 학습이 아니므로 그래프에 기록하지 않는다

- PyTorch의 합성곱도 필터를 **뒤집지 않는다** (상호상관). 우리가 배운 계산과 정확히 같다

nn.MaxPool2d: 최대 풀링

```
A = torch.tensor([[1., 2., 1., 0.],
                  [0., 1., 2., 3.],
                  [3., 0., 1., 2.],
                  [2., 4., 0., 1.]]).reshape(1, 1,
    4, 4)
print(nn.MaxPool2d(2)(A).reshape(2, 2))
```

```
tensor([[2., 3.],
        [4., 2.]])
```



- `nn.MaxPool2d(2)`: 2×2 영역마다 **최댓값** 하나. 스트라이드를 따로 주지 않으면 창 크기와 같은 2
- 학습할 매개변수가 **없다**. 채널마다 따로 계산하므로 채널 수는 그대로: $(N, 30, 24, 24) \to (N, 30, 12, 12)$

nn.Flatten: 완전연결 계층으로 넘기기

```
print(tuple(nn.Flatten()(torch.zeros(100, 30, 12, 12)).shape))
```

```
(100, 4320)
```

- 첫 번째 차원(미니배치 N)은 **남기고** 나머지를 한 줄로: $30 \times 12 \times 12 = 4320$
- `print(model)`에 나오는 `Flatten(start_dim=1, end_dim=-1)`: 1번 차원부터 끝까지 펼친다는 뜻
- 그냥 `x.reshape(-1)`로 펼치면 미니배치까지 섞여 `(432000,)`이 되어 버린다
- 이제 `nn.Linear(4320, 100)`에 넣을 수 있다

세 계층의 대응

노트의 NumPy 구현 (conv_layers.py)	PyTorch
`Convolution(W, B, stride, pad)`	`nn.Conv2d(C, FN, kernel_size, stride, padding)`
`Pooling(2, 2)`	`nn.MaxPool2d(2)`
`Flatten()`	`nn.Flatten()`

4

모델: nn.Sequential

모델 만들기

```
model = nn.Sequential(
    nn.Conv2d(1, 30, kernel_size=5),      # (N, 1, 28, 28) -> (N, 30, 24, 24)
    nn.ReLU(),
    nn.MaxPool2d(2),                      # -> (N, 30, 12, 12)
    nn.Flatten(),                         # -> (N, 4320)
    nn.Linear(30 * 12 * 12, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
).to(device)
print(model)
print("number of parameters:", sum(p.numel() for p in model.parameters()))
```

- 계층 7개를 **순서대로** 나열했을 뿐이다. `class`를 따로 만들지 않았다
- `nn.Linear(30 * 12 * 12, 100)`: 앞 계층의 출력 크기를 **직접 계산해서** 넣어야 한다
  - 필터 크기나 패딩을 바꾸면 이 숫자도 바꿔야 한다. 틀리면 실행할 때 shape 오류가 난다
- `.to(device)`: 모든 매개변수를 GPU로 (있으면)

`print(model)`의 출력

```
device: cuda
Sequential(
  (0): Conv2d(1, 30, kernel_size=(5, 5), stride=(1, 1))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (3): Flatten(start_dim=1, end_dim=-1)
  (4): Linear(in_features=4320, out_features=100, bias=True)
  (5): ReLU()
  (6): Linear(in_features=100, out_features=10, bias=True)
)
number of parameters: 433890
```

- 괄호 안의 번호 `(0)`–`(6)`: 계층의 순번. `model[0]`이 첫 번째 합성곱 계층 (뒤에서 필터를 꺼낼 때 쓴다)
- `kernel_size=(5, 5)`: 숫자 하나 5를 주면 가로세로 같은 크기로
- `MaxPool2d`의 `stride=2`: 따로 주지 않아서 창 크기와 같아졌다
- 이번 실행은 이 컴퓨터의 GPU(`cuda`)에서 했다

계층마다 모양과 매개변수 수

```
h = torch.randn(100, 1, 28, 28)
for layer in model:
    h = layer(h)
    print(type(layer).__name__,
          tuple(h.shape))
```

```
input       (100, 1, 28, 28)
Conv2d      (100, 30, 24, 24)
ReLU        (100, 30, 24, 24)
MaxPool2d   (100, 30, 12, 12)
Flatten     (100, 4320)
Linear      (100, 100)
ReLU        (100, 100)
Linear      (100, 10)
```

```
for name, p in model.named_parameters():
    print(name, tuple(p.shape), p.numel())
```

```
0.weight  (30, 1, 5, 5)        750
0.bias    (30,)                 30
4.weight  (100, 4320)       432000
4.bias    (100,)               100
6.weight  (10, 100)           1000
6.bias    (10,)                 10
total parameters: 433890
```

- `for layer in model`: `Sequential`은 계층을 하나씩 꺼낼 수 있다 → 모양을 확인하는 좋은 방법 (`cnn_torch_demo.py`)
- 매개변수의 99.6%가 `4.weight` (Flatten 다음의 Linear)에 있다. 합성곱 계층은 겨우 780개

5

학습과 평가

손실 함수, 최적화 기법, 학습, 평가

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr
     =0.001)

def train_one_epoch(model, loader):
    model.train()
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        loss = loss_fn(model(xb), yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
@torch.no_grad()
def accuracy(model, loader):
    model.eval()
    correct, n = 0, 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        correct += (model(xb).argmax(dim=1)
                    == yb).sum().item()
        n += len(yb)
    return correct / n
```

- **지난번 코드와 같다.** 모델이 CNN이 되었는데도 손실, 최적화, 학습, 평가 코드는 그대로
- `loss.backward()`: 합성곱 계층의 역전파(노트에서 `col2im`까지 직접 짰던 것)도 autograd가 해 준다
- `train_one_epoch`는 손실을 기록하지 않는다 (이번에는 검증 정확도만 본다)
- 이 모델에는 Dropout, BatchNorm이 없어 `train()`/`eval()`이 결과를 바꾸지 않지만, 습관처럼 써 둔다

학습 반복과 실행 결과

```
epochs = 10
val_acc = []
for epoch in range(1, epochs + 1):
    t0 = time.time()
    train_one_epoch(model, train_loader)
    val_acc.append(accuracy(model, val_loader))
    print(f"epoch {epoch:2d} | val acc "
          f"{val_acc[-1]:.4f} | "
          f"{time.time() - t0:.0f} s")
print(f"test acc "
      f"{accuracy(model, test_loader):.4f}")
```

```
epoch  1 | val acc 0.9754 | 4 s
epoch  2 | val acc 0.9840 | 3 s
epoch  3 | val acc 0.9826 | 3 s
epoch  4 | val acc 0.9868 | 4 s
epoch  5 | val acc 0.9860 | 3 s
epoch  6 | val acc 0.9860 | 3 s
epoch  7 | val acc 0.9880 | 3 s
epoch  8 | val acc 0.9850 | 3 s
epoch  9 | val acc 0.9888 | 3 s
epoch 10 | val acc 0.9894 | 3 s
test acc 0.9886
```

- `time.time()`: 지금 시각(초). epoch가 끝난 뒤 시각에서 빼면 걸린 시간
- 첫 epoch에 검증 97.5%, 10 epoch 뒤 test 98.86%
- 이 코드는 마지막 epoch의 모델로 test를 한 번 본다 (가장 좋은 epoch를 저장하지 않는다)

완전연결 신경망과 비교

	매개변수	epoch	test 정확도
`mnist_torch.py` (학습 관련 기술) 은닉층 4개 + BatchNorm, AdamW	110610	15	97.80%
`mnist_cnn_torch.py` 합성곱 1개 + 완전연결 2개, Adam	433890	10	98.86%

- 합성곱 계층 **하나**만 넣어도 test 오차가 2.20%에서 1.14%로 **절반 가까이** 준다
- 매개변수가 많아서가 아니다: 합성곱 계층의 매개변수는 780개뿐이다
- 합성곱은 이미지의 **공간 구조**(이웃한 픽셀)를 쓴다. 펼쳐 넣는 완전연결 신경망은 이 정보를 버린다

GPU와 CPU, 그리고 실행할 때마다 다른 결과

같은 코드, 이 컴퓨터에서	epoch당 시간	10 epoch 검증	test
GPU (RTX 3050), 1회째	약 3초	98.94%	98.86%
GPU (RTX 3050), 2회째	약 3초	98.76%	98.61%
CPU	약 10초	98.96%	98.76%

- GPU가 CPU보다 약 3배 빠르다. 코드는 **한 글자도** 바꾸지 않았다 (`device`만 달라짐)
- `torch.manual_seed(0)`을 고정했는데도 GPU에서 두 번 돌린 결과가 **다르다**
  - GPU는 수많은 덧셈을 동시에 하므로 더하는 **순서**가 실행마다 달라질 수 있고, 부동소수점에서는 순서가 바뀌면 마지막 자리가 달라진다
  - 작은 차이가 학습을 거치며 쌓인다. 정확도 0.2–0.3%p 정도의 차이는 **우연**의 범위이다
- 그래서 설정을 비교할 때는 여러 번 실행해 평균을 보는 것이 안전하다

6

학습한 필터 보기

첫 번째 합성곱 계층의 필터 꺼내기

```
# the 30 filters of the first layer: model[0].weight has shape (30, 1, 5, 5)
filters = model[0].weight.detach().cpu().numpy()
fig, axes = plt.subplots(1, 30, figsize=(12, 0.9))
for k, ax in enumerate(axes):
    ax.imshow(filters[k, 0], cmap="gray", interpolation="nearest")
    ax.set_xticks([])
    ax.set_yticks([])
fig.savefig(os.path.join(FIG_DIR, "cnn_torch_filters.png"), dpi=200, bbox_inches="tight")
plt.close(fig)
```

- `model[0]`: `Sequential`의 0번 계층 = `Conv2d`. `.weight`: $(30, 1, 5, 5)$
- `.detach()`: 기울기 추적을 끊고 → `.cpu()`: GPU에서 가져오고 → `.numpy()`: NumPy 배열로. matplotlib은 NumPy 배열을 그린다
- `filters[k, 0]`: k번째 필터의 0번 채널 → 5×5 배열 하나
- `plt.subplots(1, 30)`: 한 줄에 칸 30개. `interpolation="nearest"`: 픽셀을 흐리게 보간하지 않고 네모 그대로

학습한 필터: cnn_torch_filters.png



GPU 1회째 실행에서 학습한 첫 번째 합성곱 계층의 필터 30개 (밝을수록 큰 값)

- 밝은 부분과 어두운 부분이 나뉜 필터들: 여러 방향의 **에지**(밝기가 바뀌는 곳)에 반응한다
- 처음에는 무작위였던 숫자들이 학습을 거쳐 이런 모양이 되었다 (`nn.Conv2d`의 기본 초기화는 균등분포)
- 실행할 때마다 필터의 순서와 모양은 달라진다. 비슷한 **종류**의 필터가 나오는 것이 중요하다

7

정리와 실습

정리

이번 시간의 핵심

- 이미지 미니배치는 (N, C, H, W) 텐서: `reshape(-1, 1, 28, 28)`
- `nn.Conv2d(C, FN, kernel_size, stride, padding)`: 가중치 (FN, C, FH, FW), 편향 $(FN,)$. 출력 크기 $\frac{H+2P-FH}{S}+1$
- `nn.MaxPool2d(2)`: 2×2마다 최댓값, 매개변수 없음. `nn.Flatten()`: 미니배치 차원은 남기고 펼침
- 모델은 `nn.Sequential`로 나열. `Linear`의 입력 크기$(30\cdot 12\cdot 12)$는 직접 계산
- 손실, 최적화, 학습 루프, 평가는 완전연결 신경망과 **같다**
- 합성곱 계층 하나로 test 정확도 $97.8\% \to 98.9\%$. 같은 코드가 GPU에서 약 3배 빠르다
- GPU 결과는 실행마다 조금 다르다: 작은 차이는 우연의 범위

실습 문제

1. `nn.Conv2d(1, 30, kernel_size=5, padding=2)`로 바꾸면 `Linear`의 입력 크기는 얼마로 바꿔야 하는가? 먼저 계산하고 실행해 보자
2. 필터 수를 30에서 $10, 60$으로 바꿔 매개변수 수와 test 정확도를 비교하기
3. 합성곱 → ReLU → 풀링을 **한 번 더** 쌓기 (`nn.Conv2d(30, 60, kernel_size=3)`). 각 계층의 모양을 `for layer in model`로 확인하기
4. LeNet과 비슷한 신경망 만들기: `Conv2d(1, 6, 5, padding=2)` → `ReLU` → `MaxPool2d(2)` → `Conv2d(6, 16, 5)` → `ReLU` → `MaxPool2d(2)` → `Flatten` → `Linear(?, 120)` → `Linear(120, 84)` → `Linear(84, 10)`
5. 합성곱 뒤에 `nn.BatchNorm2d(30)`을 넣어 보기. 이번에는 `model.train()`/`model.eval()`이 결과를 바꾼다
6. 학습 뒤 test 이미지 한 장을 `model[:3]`에 넣어 특징 맵 30장 `(1, 30, 12, 12)`을 그리기 (힌트: `Sequential`도 잘라 쓸 수 있다)
7. 가장 좋은 epoch의 모델을 `copy.deepcopy(model.state_dict())`로 저장하도록 고치기